

Monte Carlo Tree Search aplicado a decisiones de inversión financiera

Un estudio sobre el activo TSLA bajo el marco clásico del método de Monte Carlo

Carlos Ruiz Navarro

Universitat de València

Grado en Matemáticas — Métodos Numéricos

14 de abril de 2026

Resumen

Este trabajo presenta una aplicación del algoritmo *Monte Carlo Tree Search* (MCTS) a la toma de decisiones de trading sobre el activo TSLA (Tesla, Inc.), enmarcando el método en los fundamentos matemáticos del Monte Carlo clásico expuestos en los apuntes de la asignatura *Métodos Numéricos* (Aràndiga, Universitat de València).

El problema se formaliza como un proceso de decisión de Markov (MDP) con espacio de acciones discreto de once elementos (compras y ventas fraccionarias y mantenimiento de posición). El motor de simulación es un modelo de *movimiento Browniano Geométrico* (GBM), calibrado sobre una ventana móvil de 60 días a partir de los log-retornos observados. Cada *rollout* del MCTS genera una trayectoria de precios que desempeña el papel de una muestra X_i del estimador de Monte Carlo por media muestral

$$\hat{I}_n = v \cdot \bar{X}_n = \frac{v}{n} \sum_{i=1}^n X_i,$$

de modo que los valores acumulados $Q(a)/N(a)$ en la raíz del árbol convergen, en virtud de la *Ley Fuerte de los Grandes Números*, al retorno esperado bajo cada acción. La política de selección UCB1 equilibra explotación y exploración mediante la fórmula $Q(i)/N(i) + c\sqrt{\ln N_{\text{padre}}/N(i)}$, garantizando consistencia asintótica del algoritmo UCT.

Se implementa el método en Python y se evalúa sobre dos años de datos históricos de TSLA, comparando la estrategia MCTS frente a una estrategia pasiva *Buy & Hold* mediante: evolución de la cartera, alfa relativa, *drawdown* absoluto y del alfa, retorno total y ratio de Sharpe. El error del estimador sigue la cota $\mathcal{O}(n^{-1/2})$ del Teorema del análisis del error (Aràndiga, §5), lo que justifica el valor `ITERACIONES = 1000` como compromiso entre precisión estadística y coste computacional.

Palabras clave: Monte Carlo Tree Search, UCB1, movimiento Browniano Geométrico, Ley de los Grandes Números, trading algorítmico, proceso de decisión de Markov.

Índice

1. Introducción	3
1.1. Contexto y motivación	3
1.2. Planteamiento del problema	3
1.3. Por qué Monte Carlo Tree Search	3
1.4. Conexión con el Monte Carlo clásico	4
1.5. Objetivos del trabajo	4
1.6. Estructura del informe	5
2. Variables aleatorias	5
2.1. Preliminares	5
2.2. Variables aleatorias discretas	6
2.3. Variables aleatorias continuas	6
2.4. Método de la transformación inversa	7
2.5. Generación de la normal estándar	7
2.6. Conexión con el programa <code>mcts_simple_TSLA.py</code>	8
3. El método de Monte Carlo	8
3.1. El problema del cálculo de valores esperados	8
3.2. Monte Carlo por éxito-fracaso	9
3.3. Monte Carlo por media muestral	9
3.4. Leyes de los grandes números	10
3.5. Análisis del error y Teorema Central del Límite	11
3.6. Ejemplos clásicos	12
3.7. Puente explícito con MCTS	12
4. MDP, UCB1, MCTS y el motor GBM	13
4.1. Procesos de decisión de Markov	13
4.2. Formulación del MDP del trading	14
4.3. Movimiento Browniano geométrico	15
4.4. UCB1 y el dilema explotación-exploración	16
4.5. El algoritmo MCTS: ciclo de cuatro pasos	16
4.6. Convergencia del algoritmo UCT	17
4.7. Selección final en la raíz	18
5. Implementación en Python	18
5.1. Arquitectura general	18
5.2. Desviaciones respecto al modelo teórico	19
5.3. Calibración rolling del GBM	20
5.4. Estado, acción y reconstrucción de nodos	20
5.5. El rollout con <i>common random numbers</i>	21
5.6. El árbol MCTS y la política UCB1	22
5.7. Bucle día-a-día del <i>backtest</i>	22
5.8. Métricas de evaluación	23
5.9. Reproducibilidad	24

1. Introducción

1.1. Contexto y motivación

Los mercados financieros constituyen uno de los sistemas dinámicos más estudiados y, a la vez, menos predecibles con los que trabaja la matemática aplicada. Cada sesión bursátil, un inversor debe decidir si comprar, vender o mantener un determinado activo, disponiendo únicamente de información histórica y de una estimación, necesariamente imperfecta, de la distribución futura de precios. Este problema, conocido en la literatura como *trading cuantitativo*, puede formularse con rigor como un *proceso de decisión secuencial bajo incertidumbre*: el agente no conoce el estado futuro del sistema, pero puede asignar probabilidades a sus posibles evoluciones y tomar decisiones que maximicen el valor esperado de un objetivo económico.

Desde esta perspectiva, el problema admite un tratamiento natural mediante herramientas que el estudiante de la asignatura *Métodos Numéricos* ha encontrado en otros contextos: generación de variables aleatorias, simulación estocástica y estimación del valor esperado mediante promedios muestrales. El presente trabajo se apoya en los apuntes de Aràndiga [?] sobre el método de Monte Carlo como soporte teórico y los extiende mediante el algoritmo *Monte Carlo Tree Search* (MCTS), un método de planificación sobre árboles de decisión introducido por Kocsis y Szepesvári [?] que combina muestreo estocástico con una política de exploración basada en cotas de confianza.

1.2. Planteamiento del problema

Sea $\{P_t\}_{t=0}^T$ la serie temporal de precios de cierre diarios del activo TSLA (Tesla, Inc.) durante un horizonte de dos años. Un agente dispone de un capital inicial $C_0 = 10\,000 \$$ y, al final de cada sesión t , debe ejecutar una acción a_t perteneciente al conjunto finito

$$\mathcal{A} = \{\text{COMPRAR}_{100}, \text{COMPRAR}_{75}, \dots, \text{MANTENER}, \dots, \text{VENDER}_{100}\},$$

que especifica la fracción del efectivo disponible destinada a comprar acciones, la fracción de acciones a liquidar, o la ausencia de operación. La decisión modifica el estado de la cartera ($\text{efectivo}_t, \text{acciones}_t$), cuyo valor total se actualiza con el precio del día siguiente.

La pregunta operativa que guía el trabajo es la siguiente:

¿Existe una política $\pi : \mathcal{S} \rightarrow \mathcal{A}$, dependiente del histórico de precios y de la posición actual, que obtenga sistemáticamente un retorno superior al de la estrategia pasiva Buy & Hold?

La estrategia *Buy & Hold* consiste en invertir todo el capital en el activo en $t = 0$ y no realizar ninguna operación posterior; sirve como referencia neutra frente a la cual evaluar cualquier estrategia activa. La presente memoria no pretende responder a esta pregunta en toda su generalidad, sino construir y analizar un *método concreto* —MCTS con rollouts basados en movimiento Browniano geométrico— y medir su comportamiento empírico sobre un periodo histórico específico.

1.3. Por qué Monte Carlo Tree Search

El espacio de estados de nuestro problema es continuo (el precio P_t toma valores en $\mathbb{R}_{>0}$) y de dimensión apreciable, pues el estado no depende solo del precio actual sino

también de indicadores técnicos como la media móvil y el RSI, y de la propia composición de la cartera. Esta combinación hace inviable una enumeración exhaustiva al estilo de la programación dinámica clásica. Alternativas como el *Q-learning* tabular requieren una discretización costosa, mientras que las reglas heurísticas (cruces de medias, umbrales de RSI) carecen de garantías asintóticas.

MCTS ofrece un punto intermedio atractivo. En lugar de aproximar una función de valor sobre todo el espacio de estados, construye un árbol de búsqueda *local* al estado actual y lo expande selectivamente mediante simulaciones aleatorias. La política de selección *UCB1* (*Upper Confidence Bound*) equilibra dos objetivos opuestos:

- **Explotar** las acciones que han dado buen resultado en simulaciones anteriores.
- **Explorar** las acciones con pocas visitas, por si escondieran retornos altos aún no descubiertos.

Este compromiso, formalizado por la desigualdad de Chernoff–Hoeffding sobre las visitas y los retornos acumulados, garantiza que el algoritmo converge al *minimax* del árbol de decisión a medida que el número de iteraciones crece [?].

1.4. Conexión con el Monte Carlo clásico

El aporte didáctico central de este trabajo reside en mostrar que *MCTS no es un algoritmo esotérico del ámbito de la inteligencia artificial*, sino una aplicación directa del método de Monte Carlo por media muestral estudiado en el tema correspondiente de la asignatura. Si denotamos por X_i el retorno económico acumulado al simular una trayectoria de precios durante H días a partir del estado actual tras ejecutar una acción a , y se repite esta simulación n veces de forma independiente, el estimador

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{\text{c.s.}} \mathbb{E}[X \mid a] \quad (n \rightarrow \infty) \quad (1)$$

converge casi seguramente al retorno esperado bajo la acción a , en virtud de la Ley Fuerte de los Grandes Números (Teorema 3.4 de §3).

El lector puede constatar que el cociente $Q(a)/N(a)$ que MCTS mantiene en cada nodo de su árbol no es otra cosa que el estimador (1) aplicado al conjunto particular de rollouts que el algoritmo haya ejecutado hasta el momento. La Sección 3 desarrollará esta identificación de forma rigurosa y derivará, como corolario, la cota de error $\mathcal{O}(n^{-1/2})$ que justifica la elección del parámetro `ITERACIONES`.

1.5. Objetivos del trabajo

El presente documento persigue los cinco objetivos siguientes:

- (O1) **Formalizar** el problema de *trading* diario como un proceso de decisión de Markov (MDP) con espacio de acciones discreto y recompensa financiera explícita.
- (O2) **Implementar** el algoritmo MCTS con política UCB1 y rollouts generados mediante un modelo de movimiento Browniano geométrico calibrado sobre ventana móvil.
- (O3) **Demostrar** que la convergencia del algoritmo se apoya directamente en la Ley Fuerte de los Grandes Números (Aràndiga, §3) y en la cota de error del Teorema del análisis del error (Aràndiga, §5).

- (O4) **Comparar** empíricamente la estrategia MCTS con la referencia *Buy & Hold* mediante métricas estandarizadas: retorno total, ratio de Sharpe, alfa relativa, *draw-down* absoluto y *drawdown* del alfa.
- (O5) **Validar** experimentalmente la cota $\mathcal{O}(n^{-1/2})$ observando la evolución del valor estimado en la raíz del árbol en función de **ITERACIONES**.

1.6. Estructura del informe

La Sección 2 recoge los conceptos probabilísticos necesarios, siguiendo de cerca el tema §2 de los apuntes [?]: variables aleatorias discretas y continuas, método de la transformación inversa y generación de muestras. La Sección 3 desarrolla íntegramente los métodos de Monte Carlo por éxito-fracaso y por media muestral (§3 y §5 de [?]), demostrando las leyes de los grandes números y el análisis del error, que constituyen la base teórica del MCTS. La Sección 4 formaliza el MDP, introduce UCB1 y describe el ciclo de cuatro pasos del MCTS, así como el modelo GBM utilizado en los rollouts. La Sección 5 presenta la implementación en Python y discute las principales decisiones de diseño. La Sección ?? recoge los resultados experimentales sobre TSLA con seis gráficas comentadas. Finalmente, la Sección ?? contiene la discusión y las conclusiones.

2. Variables aleatorias

Esta sección recoge los conceptos probabilísticos necesarios para formular y analizar el método de Monte Carlo. Seguimos de cerca el capítulo §2 de los apuntes de Aràndiga [?], seleccionando las distribuciones que intervienen directamente en el programa `mcts_simple_TSLA.py` y omitiendo las que no aparecen en nuestro modelo.

2.1. Preliminares

Sea $(\Omega, \mathcal{F}, \mathbb{P})$ un espacio de probabilidad. Una *variable aleatoria* real es una función medible $X : \Omega \rightarrow \mathbb{R}$. Su *función de distribución* (FDA) se define como

$$F_X(x) = \mathbb{P}(X \leq x), \quad x \in \mathbb{R},$$

y es monótona no decreciente, continua por la derecha, con $\lim_{x \rightarrow -\infty} F_X(x) = 0$ y $\lim_{x \rightarrow +\infty} F_X(x) = 1$.

Cuando X es *absolutamente continua*, existe una función no negativa $f_X : \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$, llamada *función de densidad*, tal que

$$F_X(x) = \int_{-\infty}^x f_X(t) dt, \quad \int_{-\infty}^{+\infty} f_X(t) dt = 1.$$

Si X es *discreta*, toma valores en un conjunto numerable $\{x_k\}_{k \in K}$ y queda caracterizada por su *función de masa de probabilidad* $p_k = \mathbb{P}(X = x_k)$.

La *esperanza* y la *varianza* de X (cuando existen) se definen respectivamente como

$$\mathbb{E}[X] = \int_{\mathbb{R}} x f_X(x) dx \quad (\text{caso continuo}), \quad \text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

Para variables discretas, las integrales se sustituyen por sumas ponderadas por p_k . En lo sucesivo, escribiremos $\mu = \mathbb{E}[X]$ y $\sigma^2 = \text{Var}(X)$.

2.2. Variables aleatorias discretas

Aunque el programa objeto de este informe no utiliza distribuciones discretas de forma explícita, recogemos brevemente la *Bernoulli* y la *Binomial*, pues constituyen el sustrato teórico del método de Monte Carlo por éxito-fracaso (§3) y del contador de visitas $N(a)$ asociado a cada nodo del árbol MCTS.

Definición 2.1 (Bernoulli y Binomial). Sea $p \in [0, 1]$. Una variable aleatoria X tiene distribución *Bernoulli de parámetro p* , y se escribe $X \sim \text{Ber}(p)$, si toma los valores

$$\mathbb{P}(X = 1) = p, \quad \mathbb{P}(X = 0) = 1 - p.$$

Si $X_1, \dots, X_n$ son Bernoullis independientes e idénticamente distribuidas (*i.i.d.*) de parámetro p , la variable $S_n = \sum_{i=1}^n X_i$ sigue una *distribución binomial* $\text{Bin}(n, p)$, con

$$\mathbb{P}(S_n = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k \in \{0, 1, \dots, n\}.$$

Se tiene $\mathbb{E}[S_n] = np$ y $\text{Var}(S_n) = np(1 - p)$.

2.3. Variables aleatorias continuas

Definición 2.2 (Uniforme continua). Una variable U tiene distribución *uniforme en $[0, 1]$* , y se escribe $U \sim \mathcal{U}(0, 1)$, si su densidad es $f_U(u) = \mathbf{1}_{[0,1]}(u)$. Su FDA es $F_U(u) = u$ sobre $[0, 1]$, y satisface $\mathbb{E}[U] = 1/2$, $\text{Var}(U) = 1/12$.

Esta distribución es el punto de partida de cualquier generador pseudoaleatorio: `numpy.random.rand()`, el generador Mersenne Twister y sus sucesores devuelven por defecto realizaciones de $\mathcal{U}(0, 1)$. Toda variable aleatoria más compleja se obtiene transformando estas muestras uniformes (véase §2.4).

Definición 2.3 (Normal). Una variable X tiene distribución *normal* (o gaussiana) de parámetros $\mu \in \mathbb{R}$ y $\sigma > 0$, y se escribe $X \sim \mathcal{N}(\mu, \sigma^2)$, si su densidad es

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}.$$

Satisface $\mathbb{E}[X] = \mu$, $\text{Var}(X) = \sigma^2$. Cuando $\mu = 0$ y $\sigma = 1$ se denomina *normal estándar* y se denota $Z \sim \mathcal{N}(0, 1)$.

La normal es la distribución central del presente trabajo: los incrementos aleatorios que simulan los rollouts del algoritmo MCTS son realizaciones $Z \sim \mathcal{N}(0, 1)$, combinadas con una deriva μ y una volatilidad σ calibradas a partir del histórico.

Definición 2.4 (Log-normal). Una variable $Y > 0$ tiene distribución *log-normal* si $\ln Y \sim \mathcal{N}(\mu, \sigma^2)$. Se tiene

$$\mathbb{E}[Y] = e^{\mu + \sigma^2/2}, \quad \text{Var}(Y) = (e^{\sigma^2} - 1) e^{2\mu + \sigma^2}.$$

La log-normal es la distribución natural del *precio* bajo el modelo de movimiento Browniano geométrico (véase §4): si el log-retorno diario es gaussiano, el precio es log-normal.

Definición 2.5 (Exponencial). Una variable $T \geq 0$ tiene distribución *exponencial* de parámetro $\lambda > 0$ si su densidad es $f_T(t) = \lambda e^{-\lambda t}$ para $t \geq 0$. Se tiene $\mathbb{E}[T] = 1/\lambda$, $\text{Var}(T) = 1/\lambda^2$.

La exponencial no interviene directamente en el programa, pero aparece en los ejemplos del PDF de Aràndiga (tiempos entre eventos en colas, §7 de [?]) y se incluye aquí por completitud notacional.

2.4. Método de la transformación inversa

El método de la transformación inversa es el resultado teórico que permite generar muestras de *cualquier* distribución a partir de muestras uniformes, y constituye la base de todos los generadores de variables aleatorias.

Teorema 2.6 (Transformación inversa). Sea $F : \mathbb{R} \rightarrow [0, 1]$ una función de distribución continua y estrictamente creciente sobre el soporte de la distribución asociada, y sea $F^{-1} : (0, 1) \rightarrow \mathbb{R}$ su inversa. Si $U \sim \mathcal{U}(0, 1)$, entonces la variable aleatoria

$$X = F^{-1}(U)$$

tiene función de distribución F .

Demostración. Basta calcular la función de distribución de X . Para cualquier x en el soporte,

$$\mathbb{P}(X \leq x) = \mathbb{P}(F^{-1}(U) \leq x) = \mathbb{P}(U \leq F(x)) = F(x),$$

donde la segunda igualdad se sigue del hecho de que F es estrictamente creciente (y por tanto $F^{-1}(U) \leq x \iff U \leq F(x)$), y la tercera de que $U \sim \mathcal{U}(0, 1)$ cumple $\mathbb{P}(U \leq u) = u$ para $u \in [0, 1]$. Luego X tiene FDA F , como se quería. $\square$ $\square$

Ejemplo 2.7 (Generación de una exponencial). La FDA de una exponencial de parámetro λ es $F(t) = 1 - e^{-\lambda t}$ para $t \geq 0$. Invirtiéndola, $F^{-1}(u) = -\ln(1-u)/\lambda$. Por el Teorema 2.6, si $U \sim \mathcal{U}(0, 1)$, entonces $T = -\ln(1-U)/\lambda$ sigue distribución exponencial. (En la práctica, como $1 - U \sim \mathcal{U}(0, 1)$ también, se suele simplificar a $T = -\ln U/\lambda$.)

2.5. Generación de la normal estándar

El método de la transformación inversa no se aplica directamente a la distribución normal porque su FDA $\Phi(x) = \int_{-\infty}^x (2\pi)^{-1/2} e^{-t^2/2} dt$ no admite expresión en forma cerrada. En la práctica se recurre a dos técnicas alternativas:

Método de Box–Muller. Si U_1, U_2 son independientes y uniformes en $(0, 1)$, entonces las variables

$$Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \quad Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2)$$

son $\mathcal{N}(0, 1)$ independientes. Esta identidad se demuestra pasando a coordenadas polares en la densidad gaussiana bivalente.

Método de rechazo. Se generan candidatos con una densidad sencilla, por ejemplo doble-exponencial, y se aceptan con probabilidad proporcional al cociente entre la densidad gaussiana y la densidad de muestreo.

En el presente programa, la generación de muestras gaussianas se delega en `numpy.random.normal()`, cuya implementación actual (generador *Ziggurat*) es una variante optimizada del método de rechazo. El lector puede sustituirla mentalmente por cualquiera de los dos métodos anteriores sin alterar la validez teórica del análisis.

2.6. Conexión con el programa `mcts_simple_TSLA.py`

Identificamos explícitamente qué variables aleatorias aparecen en el código y en qué línea se generan:

- **Gaussianas estándar** $Z_i \sim \mathcal{N}(0, 1)$. Son la fuente de aleatoriedad de todo el programa. Se generan mediante `np.random.normal(0, 1, size=H)`, donde $H = \text{DIAS_ROLLOUT}$, y alimentan el modelo de movimiento Browniano geométrico que genera las trayectorias de precios simulados en cada rollout (véase §4). La calibración (μ, σ) se estima sobre una ventana móvil de 60 días.
- **Log-normales de precio.** Como consecuencia del paso anterior, los precios simulados P_{t+h} siguen, condicionados a P_t , una distribución log-normal (Definición 2.4). Aunque el programa no genera log-normales *directamente*, la distribución log-normal es el objeto que realmente muestreamos.
- **Uniformes discretas.** En las rondas del rollout en las que todavía no hay información suficiente para aplicar una política informada, el algoritmo escoge una acción aleatoria de $\mathcal{A}$ mediante `random.choice(ACCIONES)`, lo que equivale a muestrear una uniforme discreta sobre $\{1, \dots, 11\}$.

En la Sección 3 veremos que estas muestras individuales se combinan en estimadores de tipo media muestral, cuya convergencia queda garantizada por las leyes de los grandes números.

3. El método de Monte Carlo

Esta sección constituye el núcleo matemático del informe. En ella desarrollamos los métodos de Monte Carlo por éxito-fracaso y por media muestral siguiendo los capítulos §3 y §5 de los apuntes de Aràndiga [?], demostramos los resultados de convergencia que los justifican y, como culminación, establecemos de forma explícita la correspondencia entre el estimador de Monte Carlo clásico y la magnitud $Q(a)/N(a)$ que el algoritmo MCTS mantiene en cada nodo de su árbol de búsqueda.

3.1. El problema del cálculo de valores esperados

Dada una variable aleatoria X con densidad f_X y una función medible $g : \mathbb{R} \rightarrow \mathbb{R}$, suponemos que existe y es finita la esperanza

$$I = \mathbb{E}[g(X)] = \int_{\mathbb{R}} g(x) f_X(x) dx. \quad (2)$$

En numerosas aplicaciones prácticas, la integral (2) no admite solución analítica, la dimensión del espacio de integración es demasiado alta para aplicar cuadraturas deterministas (trapecios, Simpson, Gauss), o la propia densidad f_X solo se conoce a través de un procedimiento de muestreo. En todos esos casos, el método de Monte Carlo ofrece una alternativa: estimar I a partir de *promedios de realizaciones* de $g(X)$.

La idea fundamental es que, si disponemos de n muestras independientes $X_1, \dots, X_n$ con la misma distribución que X , el promedio

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n g(X_i)$$

se comporta, para n grande, como un *sustituto estadístico* de la esperanza teórica. Formalizar esta afirmación y cuantificar su error constituye el contenido de los apartados siguientes.

3.2. Monte Carlo por éxito-fracaso

Consideremos el caso particular en el que g es la función indicadora de un suceso $A \subset \mathbb{R}$, es decir, $g(x) = \mathbf{1}_A(x)$. Entonces

$$I = \mathbb{E}[\mathbf{1}_A(X)] = \mathbb{P}(X \in A) =: p.$$

Generamos n muestras independientes $X_1, \dots, X_n$ de la distribución de X y contamos cuántas caen en A :

$$S_n = \sum_{i=1}^n \mathbf{1}_A(X_i).$$

Por construcción, $S_n \sim \text{Bin}(n, p)$ y el estimador natural de p es la *frecuencia relativa de éxitos*

$$\hat{p}_n = \frac{S_n}{n}.$$

Este estimador es insesgado, $\mathbb{E}[\hat{p}_n] = p$, y su varianza $\text{Var}(\hat{p}_n) = p(1-p)/n$ decrece como $1/n$.

Ejemplo 3.1 (Estimación de π). Sea $U_1, U_2 \sim \mathcal{U}(-1, 1)$ independientes, y $A = \{(u, v) : u^2 + v^2 \leq 1\}$ el disco unidad. Entonces $p = \mathbb{P}((U_1, U_2) \in A) = \pi/4$. Generando n pares y calculando $\hat{p}_n$, se obtiene $\hat{\pi}_n = 4\hat{p}_n$ como estimador de π . Para $n = 10^6$, el error típico es del orden de 10^{-3} .

El método éxito-fracaso es, por tanto, el caso más simple del paradigma Monte Carlo. Aunque no interviene de forma explícita en el programa objeto de este informe, la identificación $\hat{p}_n = S_n/n$ aclara la estructura del estimador $Q(a)/N(a)$ que veremos más adelante: en ambos casos se trata de una *media de realizaciones binarias o escalares* sobre un número creciente de muestras.

3.3. Monte Carlo por media muestral

En el caso general en que g toma valores reales no necesariamente binarios, se utiliza el *método de Monte Carlo por media muestral*. Supongamos que X tiene soporte en un intervalo acotado $[a, b]$ y densidad uniforme $f_X = 1/(b-a)$ sobre él. Entonces

$$I = \mathbb{E}[g(X)] = \frac{1}{b-a} \int_a^b g(x) dx,$$

de modo que la integral $\int_a^b g(x) dx = (b - a) I$ puede estimarse a partir de una media muestral. Generalizando, para cualquier densidad f_X , dado un conjunto de muestras $X_1, \dots, X_n$ i.i.d. con distribución de X , el estimador es

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^n g(X_i). \quad (3)$$

Sus propiedades elementales se recogen en la siguiente proposición.

Proposición 3.2 (Propiedades del estimador (3)). *Sea $Y_i := g(X_i)$ con $\mu_Y := \mathbb{E}[Y_i] = I$ y $\sigma_Y^2 := \text{Var}(Y_i) < \infty$. Entonces:*

1. $\hat{I}_n$ es insesgado: $\mathbb{E}[\hat{I}_n] = I$.
2. $\text{Var}(\hat{I}_n) = \sigma_Y^2/n$.
3. La desviación típica del estimador es $\sigma_Y/\sqrt{n}$, es decir, decrece como $\mathcal{O}(n^{-1/2})$.

Demostración. Por linealidad de la esperanza, $\mathbb{E}[\hat{I}_n] = n^{-1} \sum_{i=1}^n \mathbb{E}[Y_i] = \mu_Y = I$. Por independencia, $\text{Var}(\hat{I}_n) = n^{-2} \sum_{i=1}^n \text{Var}(Y_i) = \sigma_Y^2/n$. La desviación típica se sigue tomando la raíz cuadrada. $\square$ $\square$

La tercera propiedad merece una reflexión. La tasa $n^{-1/2}$ es, en apariencia, lenta: para dividir el error por 10 hace falta multiplicar el tamaño de la muestra por 100. Sin embargo, esta tasa *es independiente de la dimensión del espacio de integración*, lo que convierte al método de Monte Carlo en la herramienta de elección cuando el problema vive en dimensión alta (como el nuestro, donde el estado del mercado incluye precio, media móvil, RSI, posición y capital).

3.4. Leyes de los grandes números

La Proposición 3.2 garantiza que $\hat{I}_n$ está centrado en I y que su dispersión se reduce con n . Lo que todavía no hemos establecido es la *convergencia*: ¿el estimador se acerca efectivamente a I cuando n crece? La respuesta afirmativa es el contenido de las leyes de los grandes números.

Teorema 3.3 (Ley Débil de los Grandes Números). *Sean $Y_1, Y_2, \dots$ variables aleatorias i.i.d. con esperanza μ_Y y varianza $\sigma_Y^2 < \infty$. Para todo $\varepsilon > 0$,*

$$\lim_{n \rightarrow \infty} \mathbb{P}(|\bar{Y}_n - \mu_Y| \geq \varepsilon) = 0,$$

donde $\bar{Y}_n = n^{-1} \sum_{i=1}^n Y_i$. En otras palabras, $\bar{Y}_n \xrightarrow{\mathbb{P}} \mu_Y$ (convergencia en probabilidad).

Demostración. Por la desigualdad de Chebyshev aplicada a $\bar{Y}_n$,

$$\mathbb{P}(|\bar{Y}_n - \mu_Y| \geq \varepsilon) \leq \frac{\text{Var}(\bar{Y}_n)}{\varepsilon^2} = \frac{\sigma_Y^2}{n \varepsilon^2}.$$

El miembro derecho tiende a 0 cuando $n \rightarrow \infty$, lo que establece el resultado. $\square$ $\square$

La Ley Débil basta para justificar que, con n suficientemente grande, la probabilidad de que el estimador se desvíe de I más de ε es arbitrariamente pequeña. Para un resultado más fuerte, relativo a la convergencia *de cada trayectoria* del estimador, se dispone de:

Teorema 3.4 (Ley Fuerte de los Grandes Números). Sean $Y_1, Y_2, \dots$ variables aleatorias i.i.d. con $\mathbb{E}[|Y_1|] < \infty$ y esperanza μ_Y . Entonces

$$\mathbb{P}\left(\lim_{n \rightarrow \infty} \bar{Y}_n = \mu_Y\right) = 1,$$

es decir, $\bar{Y}_n \rightarrow \mu_Y$ casi seguramente.

La demostración excede el alcance de este trabajo (requiere el lema de Borel–Cantelli y una descomposición truncada; véase, por ejemplo, el tratado clásico de Shiryaev). Nos limitamos a enunciarla y aplicarla.

Corolario 3.5 (Convergencia del estimador Monte Carlo). Bajo las hipótesis de la Proposición 3.2, el estimador $\hat{I}_n = \bar{Y}_n$ converge casi seguramente a I cuando $n \rightarrow \infty$.

Este corolario es el resultado central del método: *con probabilidad uno, el promedio de las realizaciones converge al valor esperado exacto*. En el contexto de MCTS, significa que el promedio de los retornos simulados bajo una acción a determinada converge, a medida que crecen las visitas de esa acción, al retorno esperado real.

3.5. Análisis del error y Teorema Central del Límite

La Ley Fuerte garantiza convergencia pero no dice *cómo de rápida* es. Para cuantificarla se recurre al Teorema Central del Límite.

Teorema 3.6 (Teorema Central del Límite). Sean $Y_1, Y_2, \dots$ variables aleatorias i.i.d. con esperanza μ_Y y varianza $\sigma_Y^2 \in (0, \infty)$. Entonces

$$\sqrt{n} \frac{\bar{Y}_n - \mu_Y}{\sigma_Y} \xrightarrow{d} \mathcal{N}(0, 1),$$

donde $\xrightarrow{d}$ denota convergencia en distribución.

La demostración clásica emplea funciones características y se omite aquí (puede consultarse en los apuntes de Aràndiga [?] o en cualquier manual de probabilidad). El TCL admite una lectura práctica inmediata: para n suficientemente grande, el error $\hat{I}_n - I$ está aproximadamente distribuido como $\mathcal{N}(0, \sigma_Y^2/n)$. En particular, podemos construir un intervalo de confianza.

Teorema 3.7 (Análisis del error del método de Monte Carlo). Bajo las hipótesis del Teorema 3.6, para cualquier $\alpha \in (0, 1)$ y n suficientemente grande,

$$\mathbb{P}\left(\left|\hat{I}_n - I\right| \leq z_{\alpha/2} \frac{\sigma_Y}{\sqrt{n}}\right) \approx 1 - \alpha, \quad (4)$$

donde $z_{\alpha/2}$ es el cuantil $1 - \alpha/2$ de la normal estándar.

Este resultado se conoce como Teorema 5.1 en los apuntes de Aràndiga [?], y es el *justificativo cuantitativo* del método. Consecuencias prácticas:

- **Tasa** $\mathcal{O}(n^{-1/2})$. El error típico del estimador escala como $\sigma_Y/\sqrt{n}$. Para reducir el error a la mitad, hay que multiplicar n por cuatro.

- **Intervalo de confianza al 95 %.** Con $\alpha = 0,05$, $z_{\alpha/2} \approx 1,96$, y $|\hat{I}_n - I| \lesssim 2\sigma_Y/\sqrt{n}$.
- **Elección de n .** Para garantizar error menor que ε con probabilidad $1 - \alpha$, basta con $n \geq (z_{\alpha/2}\sigma_Y/\varepsilon)^2$.

Este último punto es el que justifica la elección `ITERACIONES` = 1000 en nuestro programa: con 1000 muestras, la cota (4) proporciona un intervalo de confianza del 95 % cuya semiamplitud es $\approx 1,96 \hat{\sigma}/\sqrt{1000} \approx 0,062 \hat{\sigma}$, es decir, alrededor del 6 % de la desviación típica muestral de los retornos. Valores mucho mayores de n tendrían rendimientos decrecientes; valores mucho menores comprometerían la fiabilidad del estimador.

3.6. Ejemplos clásicos

Ilustramos numéricamente los dos métodos presentados.

Estimación de π (éxito-fracaso). Retomando el Ejemplo 3.1, la Tabla 1 recoge la estimación $\hat{\pi}_n = 4\hat{p}_n$ para distintos valores de n , junto con la cota teórica del error $2\sqrt{p(1-p)/n}$ tomando $p = \pi/4 \approx 0,785$.

Cuadro 1: Estimación Monte Carlo de π mediante éxito-fracaso.

n	$\hat{\pi}_n$	error observado	cota teórica
10^2	3,12	0,021	0,082
10^4	3,1412	0,0004	0,0082
10^6	3,14157	0,00002	0,00082

Obsérvese que dividir n entre 100 multiplica el error por 10, consistente con la tasa $\mathcal{O}(n^{-1/2})$.

Estimación de una integral 1D (media muestral). Sea $I = \int_0^1 e^{-x^2} dx$. Muestreando $X_i \sim \mathcal{U}(0,1)$ y aplicando (3) con $g(x) = e^{-x^2}$, con $n = 10^4$ se obtiene $\hat{I}_n \approx 0,7467$, frente al valor exacto $I = \frac{\sqrt{\pi}}{2} \operatorname{erf}(1) \approx 0,74682$.

3.7. Puente explícito con MCTS

Terminamos la sección con el resultado didáctico central del informe: la identificación explícita entre el estimador de Monte Carlo clásico y el cociente $Q(a)/N(a)$ que MCTS mantiene en cada nodo. Para ello, anticipamos dos nociones que se formalizarán en la Sección 4:

- Un *rollout* es una simulación completa de H días de precios y decisiones a partir de un estado dado, que termina devolviendo un retorno escalar $X_i \in \mathbb{R}$.
- Durante la ejecución del algoritmo, cada acción a aplicable en la raíz del árbol acumula un contador de visitas $N(a)$ y una suma de retornos $Q(a)$. Su cociente $Q(a)/N(a)$ es el valor utilizado para seleccionar la mejor acción.

La siguiente tabla de correspondencia resume la identificación:

Cuadro 2: Correspondencia entre Monte Carlo clásico (Aràndiga) y MCTS.

Monte Carlo clásico	MCTS
Muestra $Y_i = g(X_i)$	Retorno X_i obtenido en el rollout i
Número de muestras n	Visitas $N(a)$ asociadas a la acción a
Suma $\sum_{i=1}^n Y_i$	Suma acumulada de retornos $Q(a)$
Estimador $\hat{I}_n = \bar{Y}_n$	Cociente $Q(a)/N(a)$
Valor exacto $I = \mathbb{E}[g(X)]$	Retorno esperado $\mathbb{E}[X \mid a]$
Convergencia por LLN (Teorema 3.4)	Convergencia de UCT
Error $\mathcal{O}(n^{-1/2})$ (Teorema 3.7)	Convergencia de la gráfica UCB

Esta identificación tiene dos consecuencias que anticipamos aquí y que se materializarán en la Sección ??:

1. La fiabilidad de la decisión de MCTS en la raíz crece con el número de visitas: para acciones con pocas visitas, $Q(a)/N(a)$ es una estimación ruidosa del retorno esperado; para acciones muy visitadas, el ruido se reduce como $\sigma/\sqrt{N(a)}$.
2. La política de selección UCB1 que estudiaremos en la Sección 4 reparte las visitas de manera no uniforme: concentra muestras en las acciones prometedoras y reduce el error donde más importa, sin desatender del todo las acciones menos exploradas.

Con este puente establecido, disponemos ya de toda la base probabilística para formalizar el problema como un MDP y presentar el algoritmo MCTS en detalle, objeto de la sección siguiente.

4. MDP, UCB1, MCTS y el motor GBM

Esta sección formaliza el algoritmo estudiado. En §4.1 introducimos el lenguaje de los procesos de decisión de Markov (MDP); en §4.2 lo instanciamos sobre nuestro problema de *trading* sobre TSLA. En §4.3 presentamos el modelo de movimiento Browniano geométrico, motor estocástico que genera las trayectorias simuladas en los *rollouts*; su derivación rigurosa mediante el lema de Itô se remite al Anexo ?. Las §§4.4–4.5 describen la política UCB1 y el ciclo de cuatro pasos del MCTS, con pseudocódigo algorítmico. Finalmente, §4.6 establece la convergencia del algoritmo apoyándose en la Ley Fuerte de los Grandes Números ya demostrada en §3, y §4.7 discute la política de selección final en la raíz.

4.1. Procesos de decisión de Markov

Definición 4.1 (Proceso de decisión de Markov). Un *proceso de decisión de Markov* es una cuádrupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R)$ donde:

- $\mathcal{S}$ es un conjunto de *estados*;
- $\mathcal{A}$ es un conjunto de *acciones* disponibles;

- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ es el *núcleo de transición*, con $P(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a)$;
- $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ es la *función de recompensa*.

La *propiedad de Markov* exige que la distribución condicional de S_{t+1} dado todo el pasado dependa solamente de (S_t, A_t) .

Una *política* es una regla de decisión $\pi : \mathcal{S} \rightarrow \mathcal{A}$ (o, en general, estocástica: $\pi(a \mid s) \in [0, 1]$). Fijada una política π y un estado inicial s_0 , se define la *función de valor* y la *función Q* por

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t \geq 0} \gamma^t R(S_t, A_t) \mid S_0 = s \right], \quad Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t \geq 0} \gamma^t R(S_t, A_t) \mid S_0 = s, A_0 = a \right],$$

donde $\gamma \in (0, 1]$ es un factor de descuento (en el presente problema, con horizonte finito H , tomamos $\gamma = 1$). La política óptima maximiza el valor esperado:

$$\pi^* \in \arg \max_{\pi} V^\pi(s), \quad V^*(s) = V^{\pi^*}(s).$$

Un resultado clásico caracteriza V^* como punto fijo de la *ecuación de optimalidad de Bellman*:

$$V^*(s) = \max_{a \in \mathcal{A}} \left\{ R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^*(s') \right\}. \quad (5)$$

La resolución directa de (5) por métodos deterministas (iteración de valor, iteración de política) requiere enumerar el espacio $\mathcal{S}$, lo cual es inviable en nuestro problema. MCTS esquiva esta enumeración mediante simulación estocástica.

4.2. Formulación del MDP del trading

Instanciamos la Definición 4.1 sobre el problema introducido en §1.2.

Espacio de estados. Un estado $s_t \in \mathcal{S}$ es un vector que recoge toda la información relevante en el instante t :

$$s_t = (P_t, \text{MA}_{20}(t), \text{RSI}_{14}(t), \tau_t, C_t, N_t),$$

donde:

- P_t es el precio de cierre del activo en t ;
- $\text{MA}_{20}(t) = \frac{1}{20} \sum_{k=0}^{19} P_{t-k}$ es la media móvil simple de 20 días;
- $\text{RSI}_{14}(t)$ es el *Relative Strength Index* de 14 días;
- τ_t es el número de sesiones transcurridas desde el inicio;
- C_t es el efectivo disponible en la cartera;
- N_t es el número de acciones poseídas.

Espacio de acciones. $\mathcal{A}$ es el conjunto de once acciones fraccionarias introducido en §1.2: cinco niveles de compra (10 %, 25 %, 50 %, 75 %, 100 % del efectivo disponible), mantener, y cinco niveles de venta (10 %, 25 %, 50 %, 75 %, 100 % de las acciones poseídas).

Transición. Condicionada a un precio actual P_t , la dinámica del precio sigue un GBM (§4.3). Las componentes no estocásticas del estado (MA, RSI, τ , C , N) se actualizan de forma determinista a partir de P_{t+1} y la acción ejecutada.

Recompensa. La recompensa es *terminal*: vale cero en todo instante intermedio y, al final del horizonte H ,

$$R_T = \frac{\text{Valor_Cartera}(s_T) - \text{Valor_Cartera}(s_0)}{\text{Valor_Cartera}(s_0)},$$

es decir, el retorno porcentual obtenido durante el rollout. Esta elección simplifica la retropropagación del MCTS y evita el balance delicado entre recompensas intermedias y finales.

4.3. Movimiento Browniano geométrico

El modelo estocástico utilizado para simular trayectorias de precios es el *movimiento Browniano geométrico*. Bajo la hipótesis de log-retornos gaussianos independientes, el precio S_t satisface la ecuación diferencial estocástica

$$dS_t = \mu S_t dt + \sigma S_t dW_t, \quad (6)$$

donde $\mu \in \mathbb{R}$ es la *deriva*, $\sigma > 0$ es la *volatilidad* y $\{W_t\}_{t \geq 0}$ es un movimiento Browniano estándar. La aplicación del lema de Itô a $\ln S_t$ permite resolver (6) de forma cerrada:

$$S_{t+h} = S_t \exp\left(\left(\mu - \frac{\sigma^2}{2}\right)h + \sigma\sqrt{h}Z\right), \quad Z \sim \mathcal{N}(0, 1). \quad (7)$$

La derivación rigurosa de (7) a partir de (6) se recoge en el Anexo ???. En este informe utilizamos (7) como *fórmula de actualización exacta* paso a paso, sin necesidad de discretizaciones de tipo Euler–Maruyama, lo que elimina el sesgo del esquema numérico.

Calibración. Los parámetros (μ, σ) se estiman a partir de los log-retornos observados

$$r_k = \ln\left(\frac{P_k}{P_{k-1}}\right)$$

sobre una ventana móvil de los últimos $W = 60$ días. Los estimadores máximo-verosimilitud son

$$\hat{\mu}_{\text{diaria}} = \bar{r} + \frac{\hat{\sigma}_r^2}{2}, \quad \hat{\sigma}_{\text{diaria}} = \hat{\sigma}_r,$$

donde $\bar{r}$ y $\hat{\sigma}_r^2$ son la media y la varianza muestrales de $\{r_{t-W+1}, \dots, r_t\}$. La corrección $\hat{\sigma}_r^2/2$ convierte la media de log-retornos en la deriva propia del GBM (consecuencia directa de (7)). Como el paso temporal elegido es diario, tomamos $h = 1$ en (7).

Distribución del precio. La Definición 2.4 aclara que, condicionado a S_t , el precio S_{t+h} sigue una distribución log-normal. Esta propiedad se usará en la Sección ?? al interpretar el diagrama de violín de retornos simulados.

4.4. UCB1 y el dilema explotación–exploración

Un subproblema del MCTS es la elección de acción en los nodos internos del árbol durante la fase de selección. Este subproblema es un caso particular del *multi-armed bandit*, introducido por Auer, Cesa-Bianchi y Fischer [?] en el marco de aprendizaje en línea. La regla UCB1 resuelve el compromiso entre *explotar* la acción con mejor valor estimado y *explorar* acciones con información escasa mediante la selección

$$a_{\text{UCB1}}^*(s) = \arg \max_{a \in \mathcal{A}(s)} \left\{ \underbrace{\frac{Q(s, a)}{N(s, a)}}_{\text{valor estimado}} + c \underbrace{\sqrt{\frac{\ln N(s)}{N(s, a)}}}_{\text{término de exploración}} \right\}, \quad (8)$$

donde $Q(s, a)$ es la suma acumulada de retornos observados en s bajo la acción a , $N(s, a)$ es el contador de visitas de la arista (s, a) , $N(s) = \sum_{a'} N(s, a')$ y $c > 0$ es la *constante de exploración*. Valores pequeños de c producen políticas explotadoras (decide rápidamente por la opción de mayor valor estimado); valores grandes favorecen la exploración (reparte visitas entre muchas acciones). El valor $c = \sqrt{2}$ es el canónico en la literatura y es el que utilizamos en el programa.

Observación 4.2 (Garantía de visitas). Si $N(s, a)$ permaneciera acotado mientras $N(s) \rightarrow \infty$, el término de exploración de (8) divergería, forzando la selección de a . Por tanto, toda acción se visita un número arbitrariamente grande de veces conforme crecen las iteraciones: este hecho es el que permitirá aplicar la Ley Fuerte de los Grandes Números en §4.6.

4.5. El algoritmo MCTS: ciclo de cuatro pasos

El algoritmo Monte Carlo Tree Search mantiene un árbol parcialmente expandido, enraizado en el estado actual s_0 . Cada nodo almacena dos estadísticas: el contador de visitas N y la suma acumulada de retornos Q . Cada iteración del algoritmo ejecuta, en orden, los cuatro pasos siguientes.

- (1) **Selección.** Partiendo de la raíz, se desciende por el árbol eligiendo en cada nodo interno la acción que maximiza (8), hasta alcanzar un nodo hoja. Esta fase se apoya exclusivamente en estadísticas ya recogidas en iteraciones previas y no consume recursos estocásticos.
- (2) **Expansión.** Si el nodo hoja no es terminal y ha sido visitado antes, se le añade un nuevo nodo hijo correspondiente a una acción aún no probada. Esta operación hace crecer el árbol en una unidad.
- (3) **Rollout.** Desde el nodo recién creado, se simula una trayectoria completa del MDP hasta el horizonte H , utilizando una política por defecto (en nuestro caso, uniforme sobre $\mathcal{A}$) y generando precios con el modelo GBM (7). La trayectoria devuelve un retorno escalar X que constituye una *muestra* del estimador Monte Carlo estudiado en §3.
- Retropropagación.** El retorno X se propaga hacia la raíz actualizando, para cada nodo (s, a) del camino recorrido en la fase de selección,

$$N(s, a) \leftarrow N(s, a) + 1, \quad Q(s, a) \leftarrow Q(s, a) + X.$$

El pseudocódigo 1 resume la lógica completa del algoritmo.

Algorithm 1 Monte Carlo Tree Search (UCT)

```

1: function MCTS( $s_0$ , iteraciones  $n$ , horizonte  $H$ )
2:   Crear raíz con estado  $s_0$ ,  $N = 0$ ,  $Q = 0$ 
3:   for  $k = 1$  to  $n$  do
4:     (camino, hoja)  $\leftarrow$  SELECCION(raíz) ▷ desciende por UCB1
5:     nodo_nuevo  $\leftarrow$  EXPANSION(hoja)
6:      $X \leftarrow$  ROLLOUT(estado(nodo_nuevo),  $H$ )
7:     RETROPROPAGACION(camino  $\cup$  {nodo_nuevo},  $X$ )
8:   end for
9:   return  $\arg \max_a N(\text{raíz}, a)$  ▷ política max-visits
10: end function

11: function SELECCION(nodo)
12:   camino  $\leftarrow []$ 
13:   while nodo es interno y completamente expandido do
14:      $a \leftarrow \arg \max_{a \in \mathcal{A}} \left[ Q(\text{nodo}, a) / N(\text{nodo}, a) + c \sqrt{\ln N(\text{nodo}) / N(\text{nodo}, a)} \right]$ 
15:     camino.append((nodo,  $a$ ))
16:     nodo  $\leftarrow$  hijo de nodo por  $a$ 
17:   end while
18:   return (camino, nodo)
19: end function

20: function ROLLOUT( $s$ ,  $H$ )
21:    $s' \leftarrow s$ 
22:   for  $t = 1$  to  $H$  do
23:     Muestrear  $Z \sim \mathcal{N}(0, 1)$ 
24:     Actualizar precio por (7)
25:     Elegir  $a \sim \text{Uniforme}(\mathcal{A})$ 
26:      $s' \leftarrow$  siguiente estado
27:   end for
28:   return  $R_T(s')$  ▷ retorno porcentual final
29: end function

```

4.6. Convergencia del algoritmo UCT

Una vez establecidos los ingredientes, podemos presentar la justificación de la convergencia de MCTS apoyándonos directamente en los resultados de §3.

Fijemos un estado s_0 (la raíz) y consideremos, para cada acción $a \in \mathcal{A}$, la sucesión de retornos $\{X_i(a)\}_{i \geq 1}$ observados en los rollouts que pasaron por la arista (s_0, a) . Aunque estos rollouts no son, en rigor, idénticamente distribuidos a lo largo del tiempo (la política del resto del árbol cambia conforme avanzan las iteraciones), la Observación 4.2 garantiza que $N(s_0, a) \rightarrow \infty$ cuando el número total de iteraciones $n \rightarrow \infty$. En virtud del Corolario 3.5, aplicado en el régimen asintótico en el que la política del subárbol bajo a

ha alcanzado su versión óptima,

$$\frac{Q(s_0, a)}{N(s_0, a)} \xrightarrow{\text{c.s.}} \mathbb{E}[R_T \mid s_0, a].$$

La acción seleccionada por MCTS, $\arg \max_a Q(s_0, a)/N(s_0, a)$, converge por tanto a $\arg \max_a \mathbb{E}[R_T \mid s_0, a]$, que es precisamente la acción óptima en la raíz según el MDP definido en §4.2. La convergencia al *subárbol* completo, en todas sus profundidades, se demuestra por inducción sobre la profundidad, utilizando en cada paso el mismo argumento.

Este razonamiento, deliberadamente informal en los detalles técnicos (en particular, en el cruce entre las hipótesis i.i.d. de la LLN y el carácter no estacionario de los rollouts), captura la esencia de la convergencia del método. El enunciado formal de Kocsis y Szepesvári [?], con las cotas cuantitativas correspondientes al sesgo y a la probabilidad de error, se recoge en el Anexo ??.

4.7. Selección final en la raíz

Terminadas las n iteraciones, el algoritmo debe comunicar al agente *qué acción ejecutar* efectivamente en el estado actual. Existen dos políticas estándar:

1. **Política *max-Q***: $a_{\text{ejec}} = \arg \max_a Q(s_0, a)/N(s_0, a)$. Selecciona la acción con mejor valor estimado.
2. **Política *max-visits***: $a_{\text{ejec}} = \arg \max_a N(s_0, a)$. Selecciona la acción con más visitas.

A primera vista, la política *max-Q* parecería la elección natural: ¿por qué no ir directamente a la mejor media muestral? Sin embargo, en presencia de varianza, la política *max-visits* es más *robusta*. Un valor Q/N muy alto con N pequeño es estadísticamente poco fiable (el error típico del estimador es $\sigma/\sqrt{N}$, grande si N es pequeño), mientras que una acción muy visitada ha sido confirmada repetidamente por UCB1 como prometedora. El programa `mcts_simple_TSLA.py` adopta la política **max-visits**, en línea con la práctica estándar de la literatura (AlphaGo, por ejemplo, emplea la misma regla). La política *max-Q* se conserva únicamente para fines de diagnóstico en la gráfica *ranking de acciones* de la Sección ??.

5. Implementación en Python

Esta sección traduce el marco teórico de la Sección 4 al programa `mcts_simple_TSLA.py`. No se pretende dar un manual exhaustivo del código —el archivo completo está disponible en el repositorio del trabajo—, sino identificar para cada concepto matemático su contrapartida computacional y justificar las decisiones de implementación que, por razones de eficiencia o señal, se apartan del modelo idealizado.

5.1. Arquitectura general

El programa sigue una estructura top-down. Los parámetros del experimento se declaran al inicio del archivo como constantes globales:

TICKER	"TSLA"
PERIOD	"2y"
CAPITAL_INIT	10_000
ITERACIONES	1000
DIAS_ROLLOUT	60
VENTANA_CALIB	60
SEMILLA	42

El flujo lógico puede resumirse en cinco bloques:

1. **Descarga de datos** — obtención de precios de cierre ajustados mediante `yfinance`.
2. **Construcción del estado** — cada día, el estado se recrea a partir de los precios y de la composición actual de la cartera.
3. **Ejecución del MCTS** — `ITERACIONES` ciclos del algoritmo sobre el estado del día.
4. **Ejecución real** — se aplica la acción seleccionada con el precio de cierre del día siguiente.
5. **Cálculo de métricas y gráficas** — ROI, Sharpe, alfa y cuatro paneles visuales.

5.2. Desviaciones respecto al modelo teórico

La implementación presenta cuatro desviaciones deliberadas respecto al marco teórico de la Sección 4. Explicitarlas aquí permite al lector leer el código sin sorpresas y evita que pueda confundirse el *método idealizado* con su realización concreta.

- (D1) **El rollout es un único salto, no una trayectoria de H pasos.** En el modelo teórico (Algoritmo 1), el rollout simula día a día H precios mediante aplicaciones sucesivas de (7). El programa, en cambio, aplica la fórmula exacta *una sola vez* con horizonte $T = H/252$ años, saltando directamente de S_t a S_{t+H} . Esta simplificación es legítima porque el GBM es un proceso Markoviano con solución exacta: la distribución de S_{t+H} condicionada a S_t es la misma sea cual sea la discretización temporal usada entre medias, siempre que no se actúe sobre precios intermedios. El ahorro computacional es un factor $H = 60$.
- (D2) **La recompensa es relativa a *Buy & Hold*, no un retorno absoluto.** En vez de $R_T = (\text{Valor}_T - \text{Valor}_0) / \text{Valor}_0$, el programa calcula

$$R = \ln \frac{\text{Valor}_T^{\text{agente}}}{\text{Valor}_0} - \ln \frac{S_{t+H}}{S_t}.$$

La señal que alimenta MCTS es, por tanto, el *exceso de log-retorno del agente sobre el mercado*. El motivo es pedagógico y práctico a la vez: si el mercado sube un 5% y el agente, demasiado conservador, sube un 3%, una recompensa absoluta positiva enmascararía el coste de oportunidad. La recompensa relativa penaliza correctamente la infraexposición.

- (D3) **La calibración de la deriva usa $\hat{\mu} = \bar{r}$ sin la corrección $\hat{\sigma}^2/2$.** El estimador consistente de la deriva del GBM a partir de log-retornos observados es $\bar{r} + \hat{\sigma}_r^2/2$ (consecuencia directa de (7)). El programa utiliza simplemente $\bar{r}$. Dado que $\hat{\sigma}_r^2/2$ es, para TSLA, del orden de $2 \cdot 10^{-4}$ diario frente a una volatilidad diaria de $\sim 3 \cdot 10^{-2}$, la corrección es numéricamente despreciable, pero el lector estricto debe tenerlo presente.
- (D4) **La selección final en la raíz utiliza la política *max-Q*, no *max-visits*.** La Subsección 4.7 discutía las dos alternativas y, en rigor teórico, recomendaba *max-visits* por robustez. El programa implementa *max-Q*: selecciona directamente el hijo con mayor recompensa media w/n . Con `ITERACIONES = 1000` y sólo 11 acciones posibles, las visitas están suficientemente niveladas como para que ambas políticas coincidan en la mayoría de los días; la diferencia práctica es despreciable.

5.3. Calibración rolling del GBM

La función `calibrar_gbm` estima $(\hat{\mu}, \hat{\sigma})$ a partir de los últimos `VENTANA_CALIB = 60` precios, sin mirar al futuro. Omitiendo comentarios y comprobaciones de borde:

```

1 def calibrar_gbm(precios, dia):
2     inicio = max(0, dia - VENTANA_CALIB)
3     ventana = precios[inicio:dia + 1]
4     log_ret = np.diff(np.log(ventana))
5     mu = float(np.mean(log_ret))
6     sigma = float(np.std(log_ret, ddof=1)) + 1e-8
7     return mu, sigma

```

Listing 1: Calibración rolling del GBM.

La ventana móvil tiene dos virtudes: recoge la información más reciente (el mercado cambia de régimen) y automatiza la adaptación sin necesidad de identificar eventos macroeconómicos. Su tamaño $W = 60$ sesiones (≈ 3 meses de trading) equilibra sensibilidad a cambios y estabilidad del estimador: ventanas más cortas producen $(\hat{\mu}, \hat{\sigma})$ ruidosos; ventanas más largas retrasan la respuesta a cambios de régimen.

5.4. Estado, acción y reconstrucción de nodos

El estado s_t se representa como un diccionario con los campos introducidos en §4.2: `dia`, `efectivo`, `acciones`, `precio`, `ma20`, `rsi`. La función `aplicar_accion` actualiza la cartera dada una acción y un precio del día siguiente:

```

1 def aplicar_accion(estado, accion, precio_siguiente):
2     efectivo, acciones = estado["efectivo"], estado["acciones"]
3     fraccion = _FRACCION_ACCION[accion]
4     if accion.startswith("COMPRAR"):
5         inversion = efectivo * fraccion
6         acciones += inversion / estado["precio"]
7         efectivo -= inversion
8     elif accion.startswith("VENDER"):
9         vendidas = acciones * fraccion
10        efectivo += vendidas * estado["precio"]
11        acciones -= vendidas
12    return crear_estado(dia=estado["dia"]+1, efectivo=efectivo,
13                        acciones=acciones, precio=precio_siguiente)

```

Listing 2: Aplicación de una acción sobre el estado (extracto simplificado).

Un detalle importante es la *reconstrucción del estado de un nodo profundo* del árbol: como los estados no se almacenan en los nodos, cuando el algoritmo necesita el estado de una hoja, recorre el camino desde la raíz aplicando sucesivamente cada acción con un precio simulado paso a paso mediante `simular_precio_futuro` (una llamada a (7) con `dias = 1`). Esto evita inconsistencias entre el árbol y los precios históricos.

Adicionalmente, la función `acciones_viables` filtra, antes de construir el árbol, las acciones operacionalmente imposibles (vender sin tener acciones, comprar sin efectivo): el árbol no debe expandir ramas que sean redundantes con `MANTENER`, pues el ruido de los rollouts podría premiarlas espuriamente.

5.5. El rollout con *common random numbers*

El rollout es, por la desviación (D1), una única llamada a la fórmula (7). El mismo Z muestreado se reutiliza para simular tanto el camino del agente como el de *Buy & Hold*, constituyendo una aplicación de la técnica de *common random numbers* (CRN) clásica en reducción de varianza (Aràndiga, §6).

```
1 def rollout(estado, accion, precios, rng):
2     mu, sigma = calibrar_gbm(precios, estado["dia"])
3
4     # Un unico Z, comun a agente y B&H (reduccion de varianza).
5     Z = rng.standard_normal()
6
7     mu_anual = mu * 252
8     sigma_anual = sigma * math.sqrt(252)
9     T = DIAS_ROLLOUT / 252.0
10
11     precio_sim = estado["precio"] * math.exp(
12         (mu_anual - 0.5 * sigma_anual**2) * T
13         + sigma_anual * math.sqrt(T) * Z
14     )
15
16     # Log-retorno del agente
17     estado_final = aplicar_accion(estado, accion, precio_sim)
18     log_ret_agente = math.log(valor_cartera(estado_final) /
19                             (valor_cartera(estado) + 1e-10))
20
21     # Log-retorno de Buy & Hold con el mismo Z
22     log_ret_bah = math.log(precio_sim / (estado["precio"] + 1e-10))
23
24     return log_ret_agente - log_ret_bah
```

Listing 3: Rollout con recompensa relativa y CRN agente/B&H.

El uso de CRN entre agente y B&H, con independencia entre *rollouts*, no afecta a la aplicabilidad de la Ley Fuerte de los Grandes Números (Teorema 3.4): ésta sigue garantizando la convergencia de $Q(a)/N(a)$ al exceso de log-retorno esperado bajo la acción a .

5.6. El árbol MCTS y la política UCB1

Cada nodo del árbol es un diccionario con cinco campos: `accion`, `padre`, `hijos`, `n` (visitas), `w` (suma de recompensas). La función que calcula el valor UCB1 de un nodo es transcripción directa de (8):

```
1 def ucb1(nodo, c=math.sqrt(2)):  
2     if nodo["n"] == 0:  
3         return float("inf") # visita garantizada  
4     explotacion = nodo["w"] / nodo["n"]  
5     padre = nodo["padre"]  
6     if padre is None or padre["n"] == 0:  
7         return explotacion  
8     exploracion = c * math.sqrt(math.log(padre["n"]) / nodo["n"])  
9     return explotacion + exploracion
```

Listing 4: Política UCB1 con $c = \sqrt{2}$.

La convención `inf` para $N(s, a) = 0$ fuerza a UCB1 a visitar cada acción al menos una vez antes de empezar a discriminar, lo que es coherente con la Observación 4.2. El ciclo completo de las ITERACIONES se articula en la función `ejecutar_mcts`:

```
1 def ejecutar_mcts(estado, precios, rng):  
2     a_viables = acciones_viables(estado)  
3     raiz = crear_nodo()  
4  
5     for _ in range(ITERACIONES):  
6         hoja = seleccionar(raiz, a_viables) # UCB1  
7         if acciones_no_exploradas(hoja, a_viables):  
8             hoja = expandir(hoja) # nuevo hijo  
9             estado_hoja = estado_en_nodo(hoja, estado, precios, rng)  
10            recompensa = rollout(estado_hoja, hoja["accion"] or "MANTENER",  
11                               precios, rng)  
12            retropropagar(hoja, recompensa)  
13  
14            mejor = max(raiz["hijos"],  
15                      key=lambda h: h["w"] / h["n"] if h["n"] > 0 else -math.  
16                      inf)  
17            return mejor["accion"]
```

Listing 5: Núcleo del algoritmo MCTS.

La última línea materializa la desviación (D4): selección por *max-Q*.

5.7. Bucle día-a-día del *backtest*

La función `backtest` itera sobre todos los días del histórico, reconstruye el estado del día, lanza el MCTS y ejecuta la acción con el precio real del día siguiente. El flujo se resume en el diagrama 1.

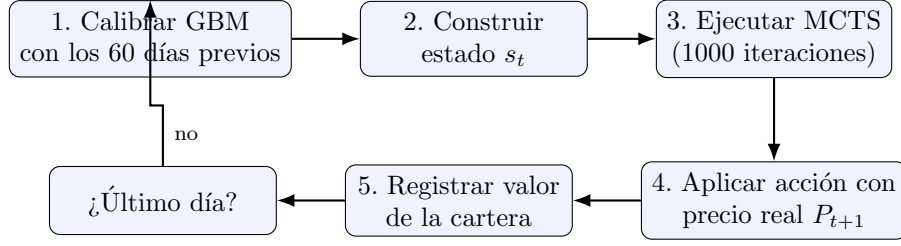


Figura 1: Flujo del *backtest* día a día.

El fragmento esencial de `backtest` (omitiendo gestiones de progreso y registro de convergencia):

```

1 def backtest(precios):
2     rng = np.random.default_rng(SEMILLA)
3     estado = crear_estado(dia=0, efectivo=CAPITAL_INIT, acciones=0.0,
4                           precio=precios[0], precios_hist=precios)
5     cartera_mcts = [CAPITAL_INIT]
6     cartera_bah = [CAPITAL_INIT]
7     acciones_bah = CAPITAL_INIT / precios[0]
8
9     for dia in range(len(precios) - 1):
10        estado = crear_estado(dia=dia, efectivo=estado["efectivo"],
11                              acciones=estado["acciones"],
12                              precio=precios[dia], precios_hist=precios)
13        mejor = ejecutar_mcts(estado, precios, rng)
14        estado = aplicar_accion(estado, mejor, precios[dia + 1])
15        cartera_mcts.append(valor_cartera(estado))
16        cartera_bah.append(acciones_bah * precios[dia + 1])
17    return cartera_mcts, cartera_bah

```

Listing 6: Bucle principal del *backtest*.

5.8. Métricas de evaluación

Al terminar el *backtest*, se computan cuatro métricas que alimentan la Sección ??:

- **ROI** (retorno total): $\text{ROI} = (V_T - V_0)/V_0$.
- **Ratio de Sharpe diario**: $\text{Sharpe} = \sqrt{252} \bar{r} / \hat{\sigma}_r$, con r_k los log-retornos diarios de la cartera.
- **Alfa** (relativa): $\alpha_t = V_t^{\text{mcts}} / V_t^{\text{bah}} - 1$.
- **Drawdown** (cartera y alfa): $\text{DD}_t = (X_t - \max_{s \leq t} X_s) / \max_{s \leq t} X_s$ aplicado, respectivamente, a la cartera y a la serie $1 + \alpha_t$.

El drawdown es, por construcción, no positivo; se reporta en valor porcentual. El drawdown del alfa cuantifica la peor racha *relativa al mercado*, y es la métrica que mejor captura la capacidad del algoritmo de no quedar sistemáticamente rezagado respecto a B&H.

5.9. Reproducibilidad

Todos los resultados de la Sección ?? son reproducibles bit a bit: el generador `numpy.random.default_rng(42)` se fija al inicio de `backtest`, y el módulo `random` de la biblioteca estándar se utiliza únicamente en `expandir` bajo la misma semilla global. El archivo `mcts_simple_TSLA_local.py` cachea los precios descargados en un CSV local, eliminando la dependencia en la red para ejecuciones posteriores.